



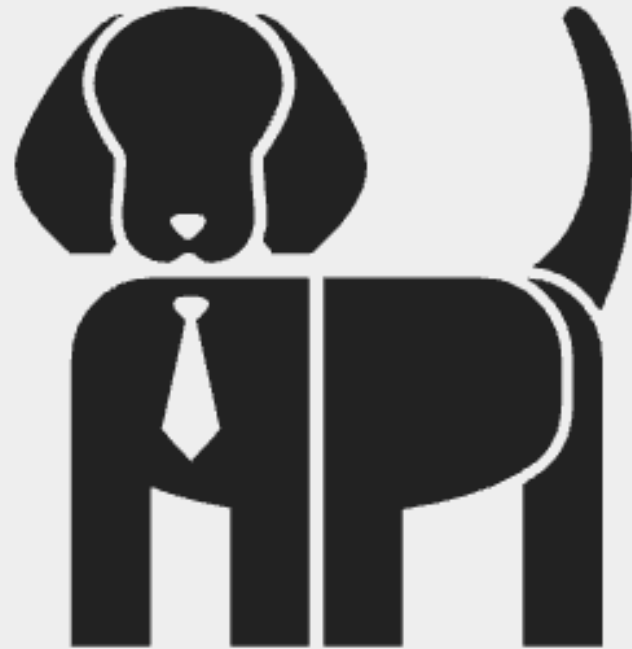
Web services

Prof. Alessandro Brawerman



DogImage

- Este projeto usa a API <https://dog.ceo/dog-api/>
- para acessar um banco de imagens de cachorros.
- Iremos passar a raça do cachorro que queremos a foto via parâmetro no request.



Dog API

DogImage APP

- Crie um novo projeto no Android Studio.
- Adicione as dependências do Retrofit no build.gradle do app.
- Adicione a dependência a biblioteca Picasso para tratarmos as imagens recebidas.
- Adicione a permissão de Internet no arquivo manifesto.

implementation("com.squareup.picasso:picasso:2.8")

- Veja um descritivo da bibliotecas em:
 - <https://square.github.io/retrofit/>
 - <https://square.github.io/picasso/>

DogImage APP

- Crie a data class para nosso Modelo

```
data class DogResponse(  
    val message: String, // A URL da imagem  
    val status: String    // O status da requisição (deve ser "success")  
)
```

- Crie a interface para API

```
interface DogApi {  
    @GET("breed/{breed}/images/random")  
    suspend fun getRandomDogImage(@Path("breed") breed: String): DogResponse  
}
```

- Note que a String recebida em breed é passada ao request.
- A URL é montada como informado pela API.

DogImage - View

<LinearLayout

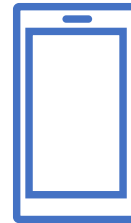
```
xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:orientation="vertical"  
    android:padding="16dp">
```

<EditText

```
    android:id="@+id/etBreed"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:hint="Digite o nome da raça"  
    android:inputType="text"  
    android:paddingTop="30dp"/>
```

<View

```
    android:layout_width="wrap_content"  
    android:layout_height="5dp"/>
```



<Button

```
    android:id="@+id/btnGetImage"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Obter Imagem"  
    android:onClick="getImage"/>
```

<View

```
    android:layout_width="wrap_content"  
    android:layout_height="10dp"/>
```

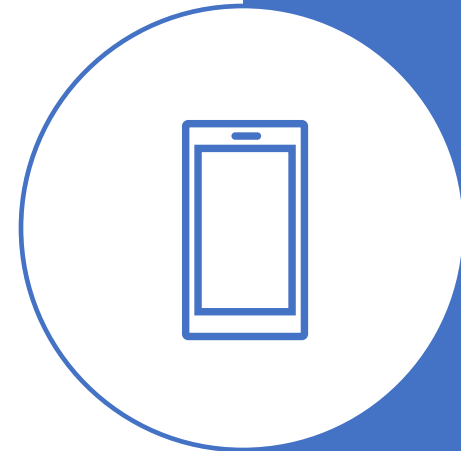
<ProgressBar

```
    android:id="@+id/progressBar"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:visibility="gone" />
```

<ImageView

```
    android:id="@+id/ivDog"  
    android:layout_width="match_parent"  
    android:layout_height="300dp"  
    android:contentDescription="Imagem do cachorro" />
```

</LinearLayout>



DogImage Controller

- Para finalizar, vamos ao Controller principal.
- Inicie com a declaração de objetos globais e inicialização no onCreate.

```
class MainActivity : AppCompatActivity() {  
  
    private lateinit var etBreed: EditText  
    private lateinit var btnGetImage: Button  
    private lateinit var ivDog: ImageView  
    private lateinit var progressBar: ProgressBar  
    private lateinit var dogApi: DogApi  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContentView(R.layout.activity_main)  
  
        etBreed = findViewById(R.id.etBreed)  
        btnGetImage = findViewById(R.id.btnGetImage)  
        progressBar = findViewById(R.id.progressBar)  
        ivDog = findViewById(R.id.ivDog)  
    }  
}
```

DogImage Controller

- Ainda no onCreate, configure o retrofit para ser usado a partir da URL base.

```
// Configura Retrofit
val retrofit = Retrofit.Builder()
    .baseUrl("https://dog.ceo/api/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()

dogApi = retrofit.create(DogApi::class.java)
} //Fim onCreate
```


DogImage Controller

- Ao clicar no botão Obter Imagem o método getImage é chamado.
- O método é responsável por preparar o request.
- Se tudo estiver certo, o método getImageApi faz é chamado para fazer o request.

```
fun getImage(view: View) {  
    val breed = etBreed.text.toString().lowercase()  
  
    if (breed.isNotEmpty()) {  
        getImageApi(breed)  
    } else {  
        Toast.makeText(context: this,  
            text: "Por favor, insira o nome de uma raça",  
            Toast.LENGTH_SHORT).show()  
    }  
}
```

getImageApi

```
// Função para buscar a imagem do cachorro usando corrotinas
private fun getImageApi(breed: String) {
    lifecycleScope.launch {
        try {
            showProgressBar()
            // Faz a requisição em segundo plano
            val response = withContext(Dispatchers.IO) {
                dogApi.getRandomDogImage(breed)
            }
            hideProgressBar()
        }
    }
}
```

getImageApi

```
// Verifica se a resposta foi bem-sucedida
if (response.status == "success") {
    // Carrega a imagem no ImageView usando Picasso
    Picasso.get().load(response.message).into(ivDog)
} else {
    Toast.makeText(context: this@MainActivity,
        text: "Raça não encontrada", Toast.LENGTH_SHORT)
        .show()
}
```

getImageApi

```
    } catch (e: Exception) {  
        Log.e( tag: "MainActivity", msg: "Erro ao buscar a imagem", e)  
        Toast.makeText(  
            context: this@MainActivity,  
            text: "Erro ao buscar a imagem. Tente novamente.",  
            Toast.LENGTH_SHORT  
        ).show()  
    }  
} //Fim corrotina  
} //Fim getImageApi
```

Métodos para mostrar e esconder o ProgressBar

```
// Função para mostrar o ProgressBar
private fun showProgressBar() {
    progressBar.visibility = View.VISIBLE
    ivDog.visibility = View.GONE
}

// Função para esconder o ProgressBar
private fun hideProgressBar() {
    progressBar.visibility = View.GONE
    ivDog.visibility = View.VISIBLE
}
```

DogImage – Execute o APP

